

Distributed Job Scheduler

Production-inspired asynchronous job scheduling platform with REST APIs, worker execution, retry handling, dead letter queue support, observability, and dashboard UI.

Backend
Python REST API

Database
SQLite relational schema

Status
Tests Passing

Deliverables Included In This PDF

Deliverable	Included	Location / Summary
Source code with setup instructions	Yes	Section 1 and project structure/setup commands.
Architecture diagram	Yes	Section 2 with component flow diagram.
ER diagram	Yes	Section 3 with relational entity mapping.
API documentation	Yes	Section 4 with endpoint tables and examples.
Design decisions document	Yes	Section 5 with major trade-offs.
Automated tests	Yes	Section 6 with test coverage and command.

1. Source Code With Setup Instructions

Project folder:
C:\Users\ASUA\Documents\Codex\2026-07-04\intern-assignment-distributed-job-scheduler-objective\outputs\distributed-job-scheduler

```
app/
  auth.py      Password hashing and bearer token authentication
  db.py        SQLite connection, schema, indexes, foreign keys
  main.py      REST API server and static dashboard server
  scheduler.py Job creation, atomic claiming, retries, DLQ, metrics
  worker.py    Polling worker with heartbeats and graceful shutdown
static/
  index.html   Web dashboard
  styles.css   Modern UI and AI animated background
  app.js       Dashboard API calls and live polling
docs/
  architecture.md
  er-diagram.md
  api.md
  design-decisions.md
tests/
  test_api.py
  test_scheduler.py
```

Run Project

```
cd C:\Users\ASUA\Documents\Codex\2026-07-04\intern-assignment-distributed-job-scheduler-objective\outputs\distributed-job-scheduler
```

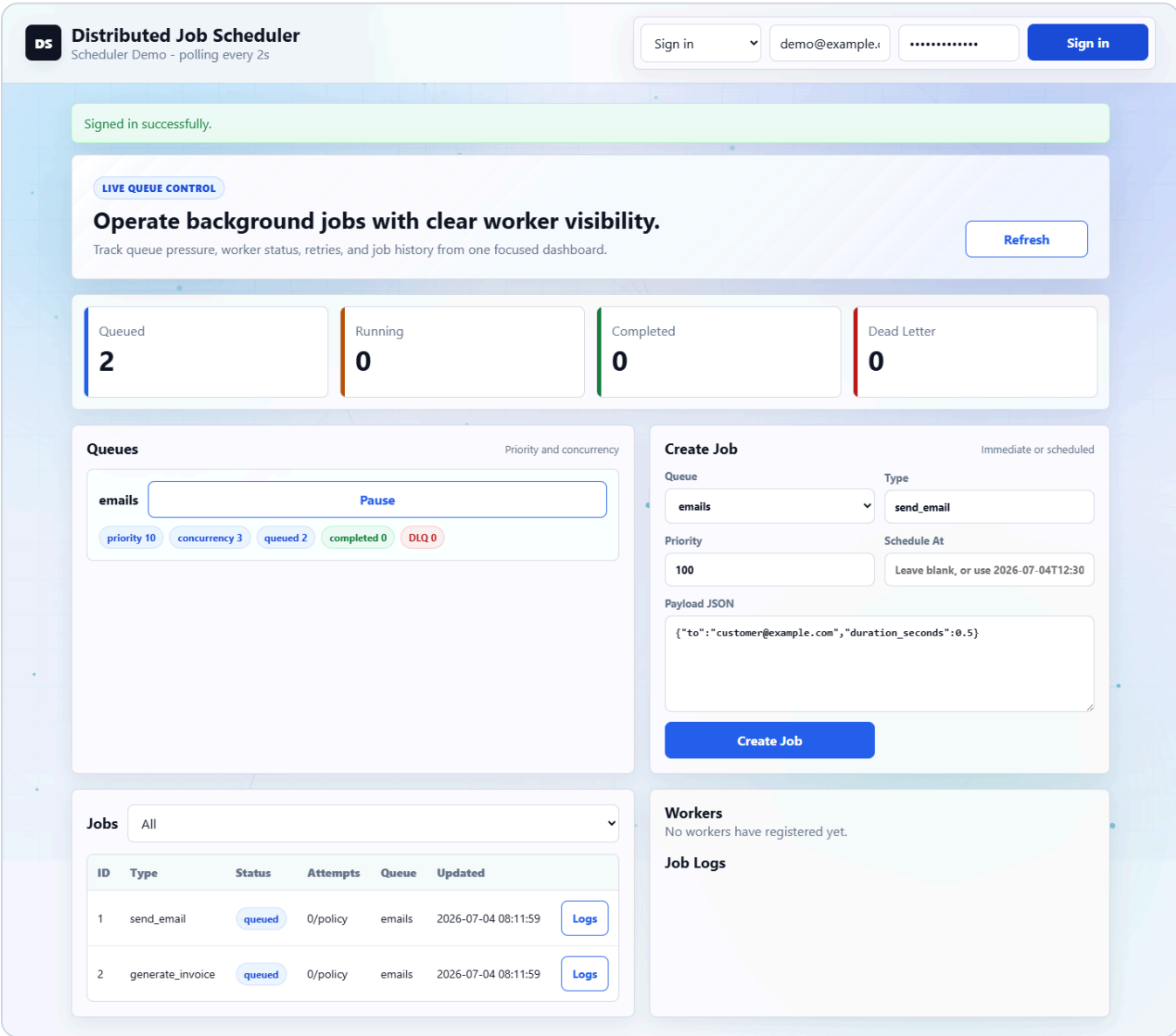
```
python -m app.main --init-db --seed
python -m app.main --serve
```

Open `http://127.0.0.1:8000` . Demo login: `demo@example.com` / `demo-password` .

Run Worker

```
python -m app.worker --worker-name worker-a
```

Dashboard Screenshot



Dashboard showing authentication, queue health, job creation, worker status, logs, and live metrics with the AI animated background UI.

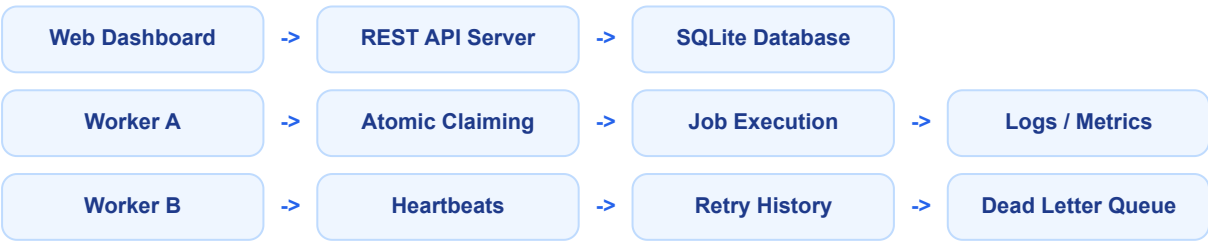
Objective

The project designs and builds a distributed job scheduling platform capable of reliably executing asynchronous background jobs across multiple workers. It evaluates backend engineering, database design, concurrency, reliability, API design, documentation, testing, and full-stack implementation.

Implemented Features

- Authentication with sign in and sign up.
- Organizations, projects, and project-scoped queues.
- Queue priority, concurrency limit, retry policy, pause/resume, and statistics.
- Immediate, delayed, scheduled, recurring cron-like, and batch jobs through REST APIs.
- Worker service with polling, atomic claim, concurrent execution, heartbeat, and graceful shutdown.
- Full lifecycle: queued, scheduled, claimed, running, completed, retry, dead letter.
- Fixed, linear, and exponential retry strategies.
- Execution logs, retry history, worker assignment, timestamps, and duration metrics.
- Responsive web dashboard with modern AI-style animated UI.

2. Architecture Diagram



The REST API owns authentication, validation, project access checks, queue configuration, job APIs, and health APIs. Workers communicate through the database, claim jobs transactionally, update executions, and emit logs/heartbeats.

Job Lifecycle

Queued -> Claimed -> Running -> Completed
Scheduled -> Queued when due
Running -> Scheduled when retryable failure occurs
Running -> Dead Letter when attempts are exhausted
Dead Letter -> Queued when manually retried

Database Design

The schema is normalized around users, organizations, projects, queues, retry policies, jobs, executions, workers, heartbeats, logs, scheduled jobs, retry history, and dead letter entries.

Table	Purpose
users	Stores user accounts and password hashes.
organizations	Groups users and projects.
projects	Owns queues and retry policies.
queues	Stores queue configuration and pause/resume state.
jobs	Stores job payload, status, schedule, attempts, and worker assignment.
job_executions	Stores each execution attempt and duration.
retry_history	Stores retry attempts, strategy, delay, and next run time.
workers	Stores worker identity, status, concurrency, and heartbeat time.
job_logs	Stores lifecycle and execution logs.

dead_letter_queue	Stores permanently failed jobs and reason.
-------------------	--

3. ER Diagram

Entity	Relationship	Related Entity
users	1 to many	auth_tokens
users	many to many through organization_members	organizations
organizations	1 to many	projects
projects	1 to many	queues
projects	1 to many	retry_policies
retry_policies	1 to many	queues
queues	1 to many	jobs
jobs	1 to many	job_executions
jobs	1 to many	retry_history
jobs	1 to many	job_logs
jobs	0 or 1 to 1	dead_letter_queue
workers	1 to many	worker_heartbeats
workers	1 to many	job_executions

Foreign keys enforce ownership boundaries and cascading behavior. Operational fields such as status, priority, schedule time, and timestamps are indexed for worker polling and dashboard filtering.

Important Indexes

- `idx_jobs_claimable` supports worker polling and atomic claiming.
- `idx_jobs_project_status` supports dashboard filtering.
- `idx_jobs_worker` supports worker assignment queries.
- `idx_executions_job` supports execution history.
- `idx_logs_job_time` supports ordered job logs.
- `idx_workers_status_heartbeat` supports worker health checks.

4. API Documentation

Area	Endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/me
Projects	GET /api/projects, POST /api/projects, GET /api/projects/{id}
Queues	GET/POST /api/projects/{id}/queues, PATCH queue, pause, resume
Jobs	GET/POST jobs, POST batch, GET logs, POST retry
Health	GET /api/projects/{id}/health, GET /api/workers

Example: Create Immediate Job

```
POST /api/projects/{project_id}/jobs
{
  "queue_id": 1,
  "type": "send_email",
  "priority": 100,
  "payload": {
    "to": "customer@example.com",
    "duration_seconds": 0.5
  }
}
```

Example: Create Scheduled Job

```
POST /api/projects/{project_id}/jobs
{
  "queue_id": 1,
  "type": "scheduled_email",
  "scheduled_at": "2026-07-04T12:30:00+05:30",
  "payload": {
    "to": "scheduled@example.com"
  }
}
```

Atomic Claiming And Reliability

Workers claim jobs inside a write transaction using `BEGIN IMMEDIATE` . The scheduler checks queue pause state, queue concurrency limit, job status, schedule time, and priority before updating the selected job to `claimed` and creating a job execution row. This prevents duplicate execution across workers.

Retry And Dead Letter Queue

Retry policies support fixed delay, linear backoff, and exponential backoff. Failed attempts are recorded in retry history. Jobs that exhaust attempts are moved to `dead_letter` and receive a dead letter queue entry containing the reason and payload snapshot.

Frontend Dashboard

The dashboard includes sign in, sign up, queue health, queue pause/resume, job creation, job explorer, worker monitor, logs viewer, retry controls, polling-based updates, and AI-style animated background UI.

5. Design Decisions And Major Trade-Offs

Decision	Reason	Production Upgrade
SQLite database	Portable, dependency-free, supports relational schema and transactions.	Move to PostgreSQL with FOR UPDATE SKIP LOCKED.
Polling workers	Simple, reliable, testable, works without external brokers.	Use Kafka, RabbitMQ, SQS, or PostgreSQL LISTEN/NOTIFY.
JSON job payload	Different job types need flexible payload shapes.	Keep metadata relational; validate payload by job type.
Bearer token auth	Simple API authentication for assignment scope.	Add refresh tokens, RBAC policies, and audit logs.
Polling dashboard	Easy live updates without extra infrastructure.	Use WebSockets for real-time streaming.

6. Automated Tests For Critical Functionality

Automated tests cover atomic claim behavior, duplicate claim prevention, dead letter queue transition, queue statistics, signup, duplicate signup conflict, login, and job creation API flow.

```
python -m unittest discover -s tests
Ran 6 tests
OK
```

Requirement Mapping

Requirement	Status
Authentication and project management	Implemented
Queue configuration	Implemented
Immediate, delayed, scheduled, recurring, batch jobs	Implemented
Worker polling, claiming, heartbeats, graceful shutdown	Implemented
Retries and dead letter queue	Implemented
Logs, retry history, metrics	Implemented
Dashboard	Implemented
Documentation and tests	Implemented